

AA2. Agregación y Composición

Autor: Pol Valle

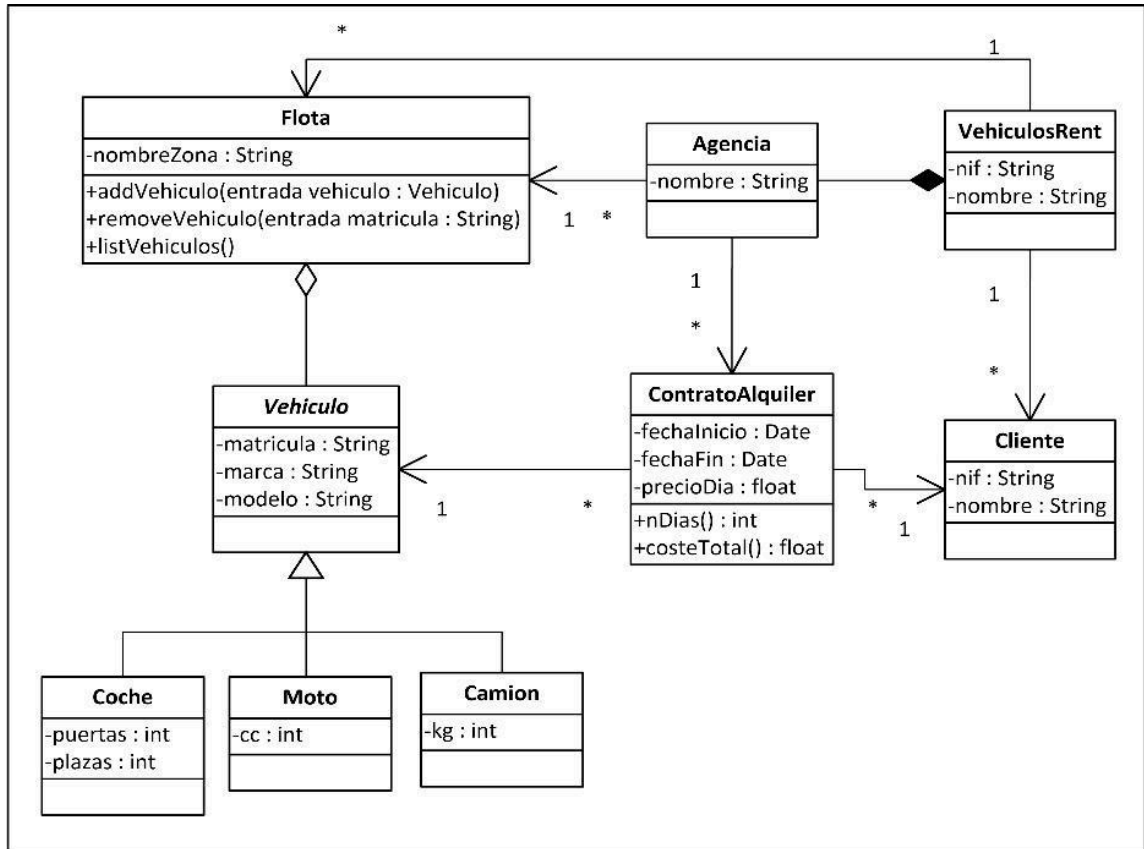
Año: 2026

Índice

Contenido

Preguntas	3
1. En el diagrama de clases la clase Flota se relaciona con las clases Agencia y VehiculosRent. ¿Por qué según el diagrama las flotas lo son de la empresa y no de las agencias o de la empresa y las agencias a la vez? Razona la respuesta.	3
2. ¿Qué cambios se deberían realizar en el diagrama de clases y en el código en Java para que un contrato de alquiler pudiera contemplar que en un mismo contrato se pudieran alquilar varios vehículos a la vez?	4
Desarrollo	4
Video	8
Autoevaluación	8

Preguntas



1. En el diagrama de clases la clase **Flota** se relaciona con las clases **Agencia** y **VehiculosRent**. ¿Por qué según el diagrama las flotas lo son de la empresa y no de las agencias o de la empresa y las agencias a la vez? Razona la respuesta.

Al ser la Agencia una composición de la empresa (**VehiculosRent**) su existencia depende de esta.

Si la agencia desapareciese la empresa y la flota seguirán existiendo, por eso al vincular la flota a la empresa y no a la agencia, nos aseguramos de que si desapareciese la agencia, seguimos disponiendo de la flota

2.¿Qué cambios se deberían realizar en el diagrama de clases y en el código en Java para que un contrato de alquiler pudiera contemplar que en un mismo contrato se pudieran alquilar varios vehículos a la vez?

En el diagrama UML deberíamos establecer una relación de agregación entre **ContratoAlquiler** y **Vehiculo**, modificar su relación (de 1 a muchos).

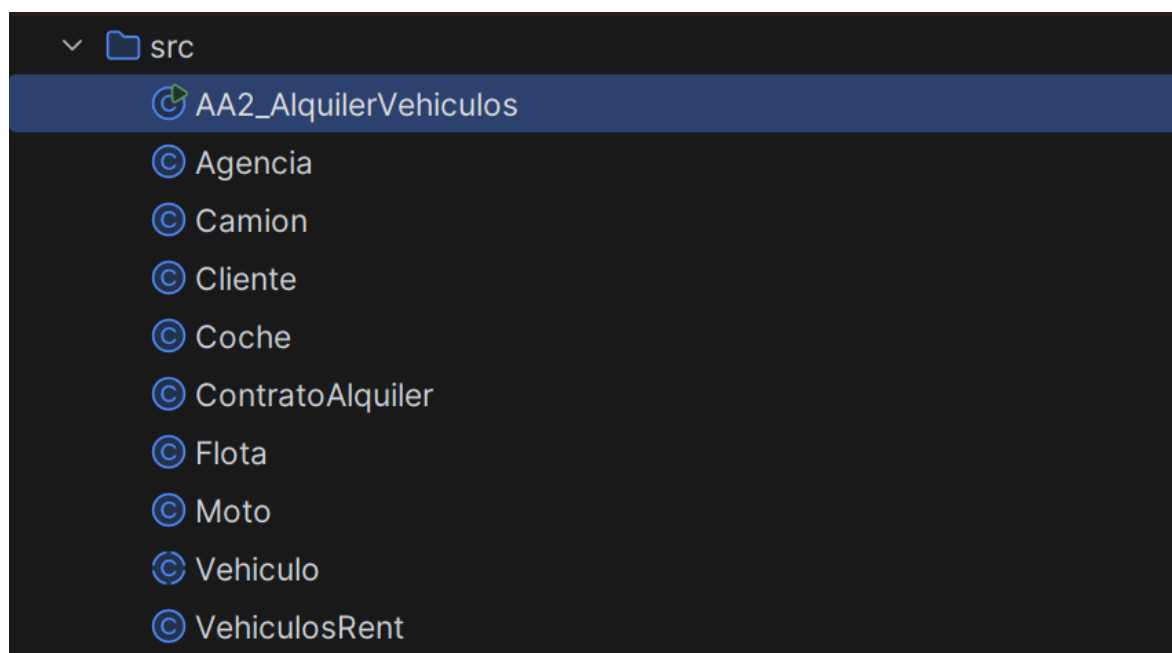
Esto refleja que los vehículos tienen una existencia independiente, así no desaparecen al finalizar el un contrato.

En Java la adaptación requeriría cambiar la variable de tipo Vehículo en la clase ContratoAlquiler por una estructura de datos dinámica, como un ArrayList. De este modo, tendríamos la capacidad de almacenar y gestionar varios vehículos simultáneamente en un único contrato."

Desarrollo

Para hacer esta actividad, he usado el código de la AA1 como base, creando un programa interactivo en consola.

He creado las nuevas clases **Flota**, **Agencia** y **VehiculosRent** implementando las relaciones de agregación y composición usando ArrayList



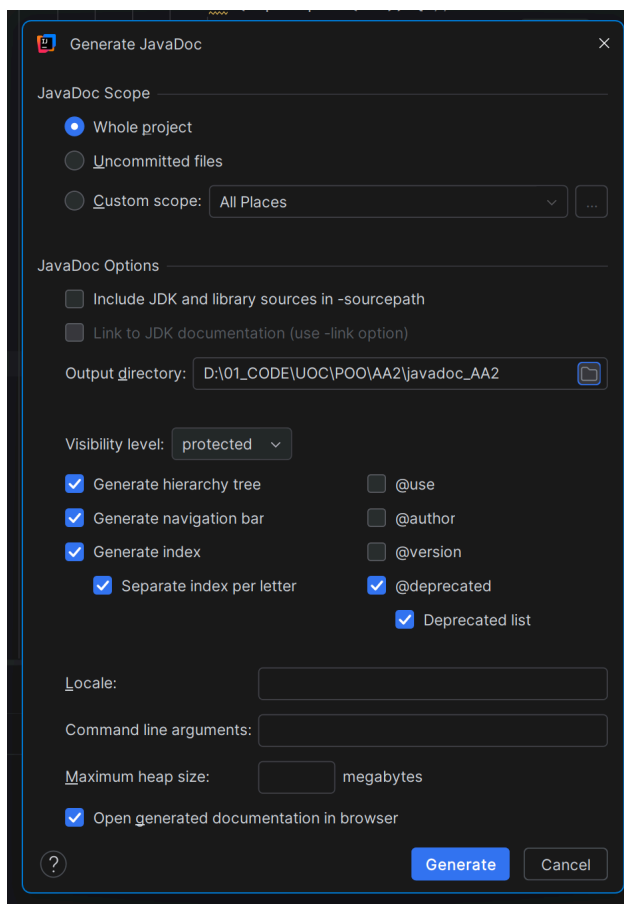
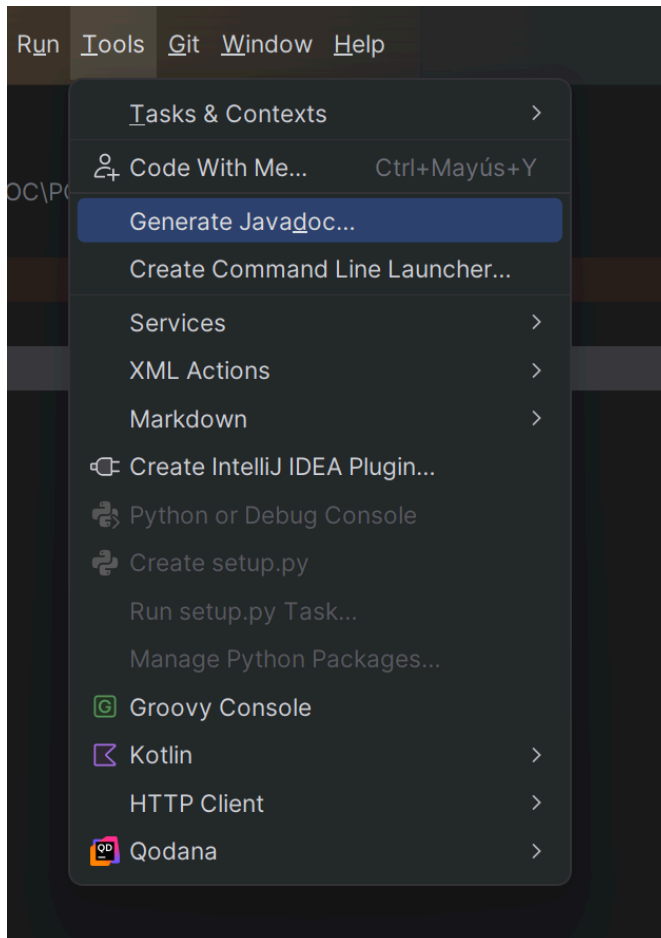
En cuanto a la lógica de gestión, me he implementar los métodos principales dentro de la clase Flota para manipular el ArrayList de vehículos:

```
/**
 * Añade un nuevo vehículo a la lista de la flota.
 * @param entradaVehiculo El objeto Vehiculo (Coche, Moto o Camion) que se va a añadir.
 */
public void addVehiculo(Vehiculo entradaVehiculo) { 6 usages 2 Doplax
    if (entradaVehiculo != null) {
        vehiculos.add(entradaVehiculo);
        System.out.println("Vehículo añadido: " + entradaVehiculo.getMatricula());
    }
}
```

```
/**
 * Recorre la lista de vehículos de la flota y los imprime por consola.
 * Si la flota está vacía, avisa al usuario.
 */
public void listVehiculos() { 1 usage 2 Doplax
    if (vehiculos.isEmpty()) {
        System.out.println("La flota " + nombreZona + " no tiene vehículos.");
        return;
    }
    System.out.println("Vehículos en la flota '" + nombreZona + "':");
    for (Vehiculo v : vehiculos) {
        System.out.println(v);
    }
}
```

```
/**
 * Busca un vehículo por su matrícula y lo elimina de la flota si lo encuentra.
 * @param matricula La matrícula del vehículo a eliminar.
 * @return true si se ha eliminado correctamente, false si no se encontró.
 */
public boolean removeVehiculo(String matricula) { 1 usage 2 Doplax
    if (matricula == null) return false;
    for (int i = 0; i < vehiculos.size(); i++) {
        Vehiculo v = vehiculos.get(i);
        if (matricula.equalsIgnoreCase(v.getMatricula())) {
            vehiculos.remove(i);
            System.out.println("Vehículo eliminado: " + matricula);
            return true;
        }
    }
    System.out.println("No se encontró vehículo con matrícula: " + matricula);
    return false;
}
```

Tambien he generado la documentación de Java Docs



All Classes
AA2_AlquilerVehiculos
Agencia
Camion
Cliente
Coche
ContratoAlquiler
Flota
Moto
Vehiculo
VehiculosRent

PACKAGE
CLASS
TREE
DEPRECATED
INDEX
HELP

PREV CLASS
NEXT CLASS
FRAMES
NO FRAMES

SUMMARY: NESTED | FIELD | CONSTR | METHOD
DETAIL: FIELD | CONSTR | METHOD

Class AA2_AlquilerVehiculos

java.lang.Object
AA2_AlquilerVehiculos

```
public class AA2_AlquilerVehiculos
extends java.lang.Object
```

Constructor Summary

Constructors

Constructor and Description
AA2_AlquilerVehiculos()

Method Summary

All Methods
Static Methods
Concrete Methods

Modifier and Type	Method and Description
static void	main(java.lang.String[] args)

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

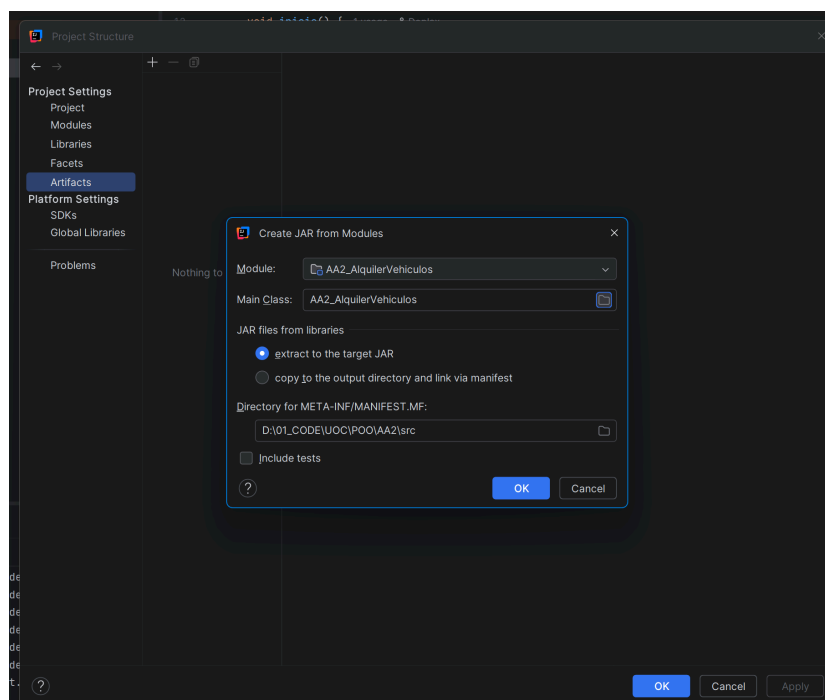
Constructor Detail

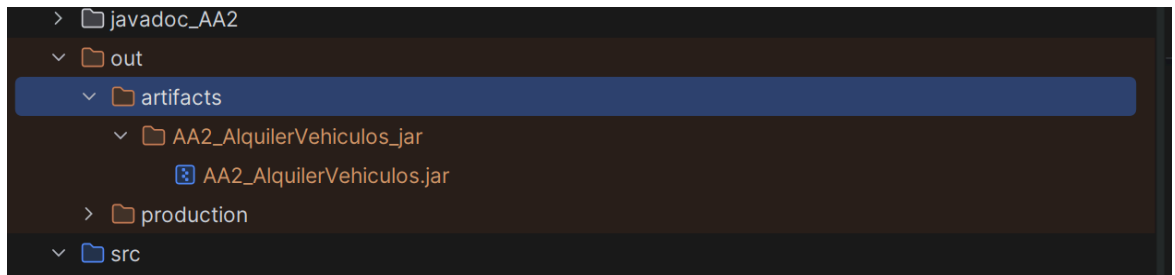
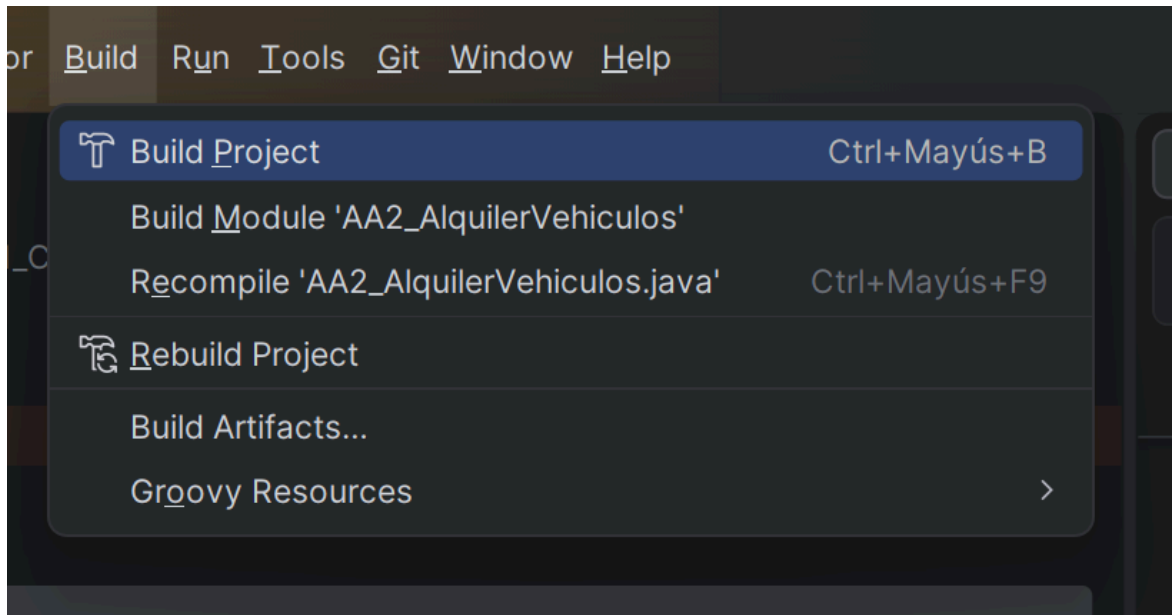
AA2_AlquilerVehiculos
public AA2_AlquilerVehiculos()

Method Detail

main
public static void main(java.lang.String[] args)

y he generado el .jar





Video

<https://youtu.be/dmQIQf1oJzc>

Autoevaluación

A nivel teórico, ya estaba familiarizado con los conceptos de Programación Orientada a Objetos que exige la práctica, me ha ayudado a recordar la diferencia entre composición y agregación.

También me ha ayudado a conocer mejor la sintaxis de java, ya que no la tengo tan por la mano como otros lenguajes.

Teniendo en cuenta la rúbrica de evaluación, considero que he cumplido con todos los requisitos: ejecución sin errores, conocimiento de la sintaxis, definición correcta de clases, interpretación del UML y generación de Javadoc.